Exploring and comparing different LLMs

Introduction

This lesson will cover:

- Different types of LLMs in the current landscape.
- Testing, iterating, and comparing different models for your use case in Azure.
- How to deploy an LLM.

Learning Goals

After completing this lesson, you will be able to:

- Select the right model for your use case.
- Understand how to test, iterate, and improve the performance of your model.
- Know how businesses deploy models.

Understand different types of LLMs

LLMs can have multiple categorizations based on their architecture, training data, and use case. Understanding these differences will help our startup select the right model for the scenario, and understand how to test, iterate, and improve performance.

There are many different types of LLM models, your choice of model depends on what you aim to use them for, your data, how much you're ready to pay and more.

Depending on if you aim to use the models for text, audio, video, image generation and so on, you might opt for a different type of model.

- **Audio and speech recognition**. Whisper-style models are still useful general-purpose speech recognition models, but production choices now also include newer speech-to-text models such as `gpt-4o-transcribe`, `gpt-4o-mini-transcribe`, and diarization variants. Evaluate language coverage, diarization, real-time support, latency, and cost for your scenario. Learn more in the OpenAI speech-to-text documentation.
- **Image generation**. DALL-E and Midjourney are well-known image generation options, but current OpenAI image APIs center on GPT Image models such as `gpt-image-2`, while Stable Diffusion, Imagen, Flux, and other model families are also common choices. Compare prompt adherence, editing support, style control, safety requirements, and licensing. Learn more in the OpenAI image generation guide and Chapter 9 of this curriculum.

- **Text generation**. Text models now span frontier models, reasoning models, smaller low-latency models, and open-weight models. Current examples include OpenAI GPT-5.x models, Anthropic Claude 4.x models, Google Gemini 3.x models, Meta Llama 4 models, and Mistral models. Do not choose only by release date or price; compare task quality, latency, context window, tool use, safety behavior, regional availability, and total cost. The Microsoft Foundry model catalog is a good place to compare models available on Azure.
- **Multi-modality**. Many current models can process more than text. Some accept image, audio, or video inputs; some can call tools; and specialized models can generate images, audio, or video. For example, current OpenAI models support text and image input, Gemini models can support text, code, image, audio, and video inputs depending on the variant, and Llama 4 Scout and Maverick are open-weight natively multimodal models. Always check each model card for supported input and output modalities before building a workflow around it.

Foundation Models versus LLMs

The term Foundation Model was coined by Stanford researchers and defined as an AI model that follows some criteria, such as:

- **They are trained using unsupervised learning or self-supervised learning**, meaning they are trained on unlabeled multi-modal data, and they do not require human annotation or labeling of data for their training process.
- **They are very large models**, based on very deep neural networks trained on billions of parameters.
- **They are normally intended to serve as a 'foundation' for other models**, meaning they can be used as a starting point for other models to be built on top of, which can be done by fine-tuning.

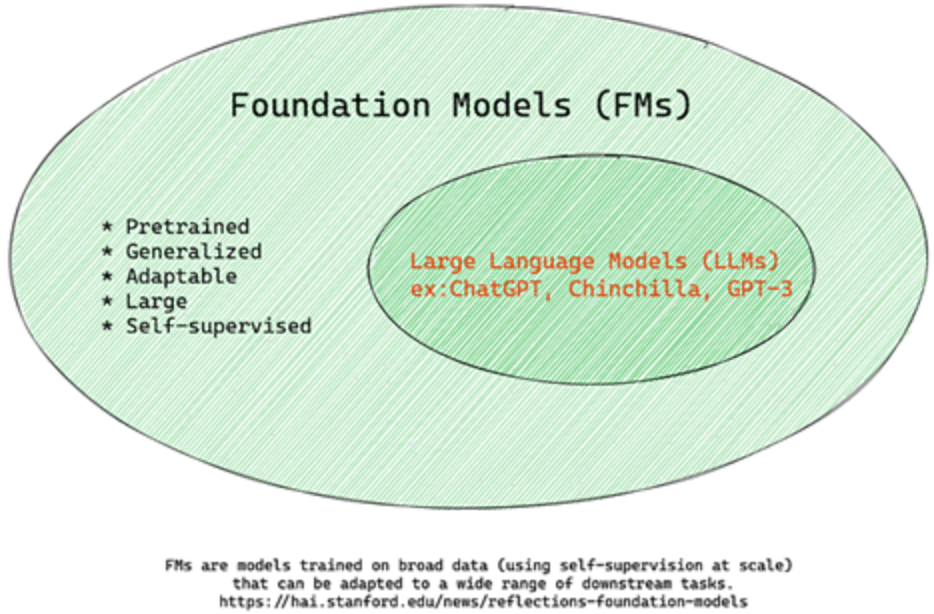


Image source: Essential Guide to Foundation Models and Large Language Models | by Babar M Bhatti | Medium

To further clarify this distinction, let's take ChatGPT as a historical example. Early versions of ChatGPT used GPT-3.5 as a foundation model. OpenAI then used chat-specific data and alignment techniques to create a tuned version that performed better in conversational scenarios, such as chatbots. Modern AI services often route between several model variants, so the service name and the underlying model name are not always the same thing.



Open-Weight/Open-Source versus Proprietary Models

Another way to categorize LLMs is whether they are open-weight, open-source, or proprietary.

Open-source and open-weight models make model artifacts available for inspection, download, or customization, but their licenses differ. Some are fully open source, while others are open-weight models with usage restrictions. They can be useful when a business needs more control over deployment, data locality, cost, or customization. However, teams still need to review license terms, serving costs, maintenance, security updates, and evaluation quality before using them in production. Examples include Meta Llama 4, some Mistral models, and many models hosted on Hugging Face.

Proprietary models are owned and hosted by a provider. These models are often optimized for managed production use and can offer strong support, safety systems, tool integration, and scale. However, customers usually cannot inspect or modify the model weights, and they must review provider terms for privacy, retention, compliance, and acceptable use. Examples include OpenAI models, Google Gemini, and Anthropic Claude.

Embedding versus Image generation versus Text and Code generation

LLMs can also be categorized by the output they generate.

Embeddings are a set of models that can convert text into a numerical form, called embedding, which is a numerical representation of the input text. Embeddings make it easier for machines to understand the relationships between words or sentences and can be consumed as inputs by other models, such as classification models, or clustering models that have better performance on numerical data. Embedding models are often used for transfer learning, where a model is built for a surrogate task for which there's an abundance of data, and then the model weights (embeddings) are re-used for other downstream tasks. An example of this category is OpenAI embeddings.

Converter

Image generation models are models that generate images. These models are often used for image editing, image synthesis, and image translation. Image generation models are often trained on large datasets of images, such as LAION-5B, and can be used to generate new images or to edit existing images with inpainting, super-resolution, and colorization techniques. Examples include GPT Image models, Stable Diffusion models, and Imagen models.

Text and code generation models are models that generate text or code. These models are often used for text summarization, translation, and question answering. Text generation models are often trained on large datasets of text, such as BookCorpus, and can be used to generate new text, or to answer questions. Code generation models, like CodeParrot, are often trained on large datasets of code, such as GitHub, and can be used to generate new code, or to fix bugs in existing code.

Encoder-Decoder versus Decoder-only

To talk about the different types of architectures of LLMs, let's use an analogy.

Imagine your manager gave you a task for writing a quiz for the students. You have two colleagues; one oversees creating the content and the other oversees reviewing them.

The content creator is like a decoder-only model: they can look at the topic, see what you already wrote, and then continue generating content based on that context. They are very good at writing engaging and informative content, but they are not always the best choice when the task is only to classify, retrieve, or encode information. Examples of decoder-only model families include GPT and Llama models.

The reviewer is like an Encoder only model, they look at the course written and the answers, noticing the relationship between them and understanding context, but they are not good at generating content. An example of Encoder only model would be BERT.

Imagine that we can have someone as well who could create and review the quiz, this is an

Service versus Model

Now, let's talk about the difference between a service and a model. A service is a product that is offered by a Cloud Service Provider, and is often a combination of models, data, and other components. A model is the core component of a service, and is often a foundation model, such as an LLM.

Services are often optimized for production use and are often easier to use than models, via a graphical user interface. However, services are not always available for free, and may require a subscription or payment to use, in exchange for leveraging the service owner's equipment and resources, optimizing expenses and scaling easily.

Models are the neural network artifacts: parameters, weights, architecture, tokenizer, and supporting configuration. Running a model locally or in a private environment requires suitable hardware, serving infrastructure, monitoring, and either a compatible open-source/open-weight license or a commercial license. Open-weight models such as Llama 4 or Mistral models can be self-hosted, but they still require computational power and

How to test and iterate with different models to understand performance on Azure

- Find the foundation model of interest in the catalog, including models sold by Azure and models from partners and community providers. Users can filter by task, provider, license, deployment option, or name.
- Review the model card, including a detailed description of intended use and training data, code samples and evaluation results on the internal evaluations library.
- Compare benchmarks across models and datasets available in the industry to assess which one meets the business scenario, through the Model Benchmarks pane.
- Fine-tune supported models on custom training data to improve model performance in a specific workload, leveraging the experimentation and tracking capabilities of Microsoft Foundry.

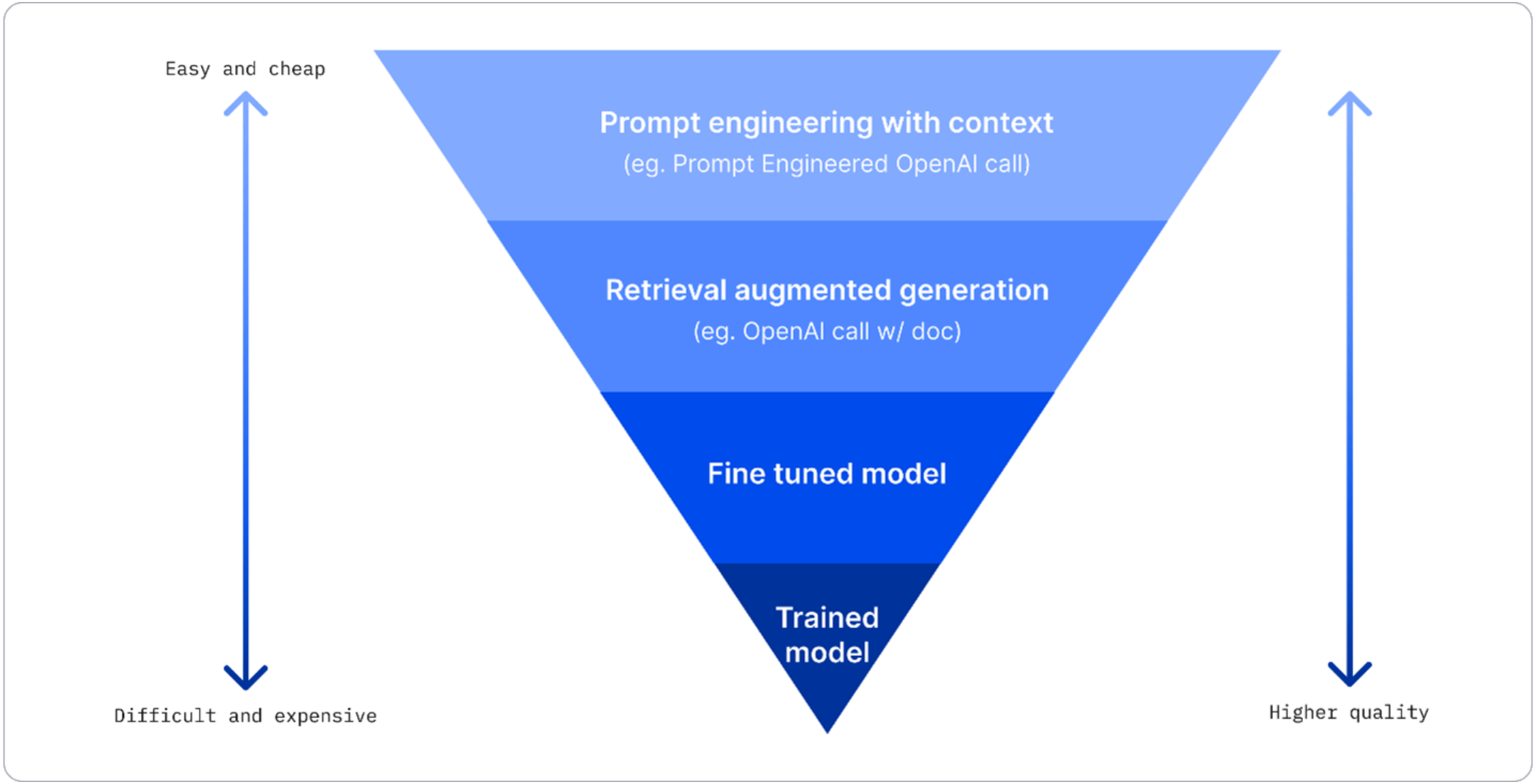
Improving LLM results

We've explored with our startup team different kinds of LLMs and a cloud platform (Microsoft Foundry) that enables us to compare different models, evaluate them on test data, improve performance, and deploy them on inference endpoints.

But when shall they consider fine-tuning a model rather than using a pre-trained one? Are there other approaches to improve model performance on specific workloads?

There are several approaches a business can use to get the results they need from an LLM. You can select different types of models with different degrees of training when deploying an LLM in production, with different levels of complexity, cost, and quality. Here are some different approaches:

- **Prompt engineering with context**. The idea is to provide enough context when you prompt to ensure you get the responses you need.
- **Retrieval Augmented Generation, RAG**. Your data might exist in a database or web endpoint for example, to ensure this data, or a subset of it, is included at the time of prompting, you can fetch the relevant data and make that part of the user's prompt.
- **Fine-tuned model**. Here, you trained the model further on your own data which led to the model being more exact and responsive to your needs but might be costly.



Img source: Four Ways that Enterprises Deploy LLMs | Fiddler AI Blog

Prompt Engineering with Context

Pre-trained LLMs work very well on generalized natural language tasks, even by calling them with a short prompt, like a sentence to complete or a question – the so-called “zero-shot” learning.

However, the more the user can frame their query, with a detailed request and examples – the Context – the more accurate and closest to user’s expectations the answer will be. In this case, we talk about “one-shot” learning if the prompt includes only one example and “few shot learning” if it includes multiple examples.
Prompt engineering with context is the most cost-effective approach to kick-off with.

Retrieval Augmented Generation (RAG)

LLMs have the limitation that they can use only the data that has been used during their training to generate an answer. This means that they don’t know anything about the facts that happened after their training process, and they cannot access non-public information (like company data).
This can be overcome through RAG, a technique that augments prompt with external data in the form of chunks of documents, considering prompt length limits. This is supported by Vector database tools (like Azure Vector Search) that retrieve the useful chunks from varied pre-defined data sources and add them to the prompt Context.

This technique is very helpful when a business doesn’t have enough data, enough time, or resources to fine-tune an LLM, but still wishes to improve performance on a specific workload and reduce risks of hallucinated, outdated, or unsupported answers.

Fine-tuned model

Fine-tuning is a process that leverages transfer learning to ‘adapt’ the model to a downstream task or to solve a specific problem. Differently from few-shot learning and RAG, it results in a new model being generated, with updated weights and biases. It requires a set of training examples consisting of a single input (the prompt) and its associated output (the completion).

This would be the preferred approach if:

- **Using smaller task-specific models**. A business would like to fine-tune a smaller model for a narrow task rather than repeatedly prompt a larger frontier model, resulting in a more cost-effective and faster solution.
- **Considering latency**. Latency is important for a specific use-case, so it’s not possible to use very long prompts or the number of examples that should be learned from the model doesn’t fit with the prompt length limit.
- **Adapting stable behavior**. A business has many high-quality examples and wants the model to consistently follow a task pattern, output format, tone, or domain-specific style. If the main problem is fresh facts or private knowledge that changes often, use RAG instead of relying on fine-tuning alone.

Trained model

Training an LLM from scratch is without a doubt the most difficult and the most complex approach to adopt, requiring massive amounts of data, skilled resources, and appropriate computational power. This option should be considered only in a scenario where a business has a domain-specific use case and a large amount of domain-centric data.